Capstone Assignment

Date
DELTA Pg No.

Case study 3 :- Performance Improvement of a college ERP Server

1. A college Erp server handles activities like admissions, fee management, attendance, timetable generation, examination processing and result publication. During peak periods like admissions and result processing, a very large no of students, faculty and administrators access the system at the same time. As a result the ERP server becomes slow, response time increases, and user experience degrades.

The management wants to improve performance without replacing the whole system. Therefore instead of a complete system redesign, it is better to identify the major performance bottlenecks and improve most critical parts of the processor-memory system.

In computer Organisation and Architecture, performance can be improved through better processor design, instruction pipelining & cache memory enhancement and memory-access optimization. A balanced approach is needed.

2. Concepts from COA used in the solution.

Amdahl's law
Processor performance.
Instruction pipelining
Cache memory.
Data hazards and control hazards.

Memory hierarchy optimization

3. Amdahl's law and its Role in Performance Improvement.

Amdahls law states that the overall speed of a system depends on the fraction of execution time improved by an enhancement.

$$\text{Speedup}_{(overall)} = \frac{1}{(1-P) + \frac{P}{S}}$$

P = fraction of work affected by the improvement
S = speed up of improved part.

This law is useful for ERP performance decisions because it prevents wasting money on components that do not contribute much to total execution time.

Example

Suppose 70% of ERP execution time is spent on memory access and cache miss.

30% is spent in processor execution

memory spent is improved by 2 times

$$\text{Speedup}_{(overall)} = \frac{1}{(1-0.7) + \frac{0.7}{2}} = \frac{1}{0.3 + 0.35} = 1.54$$

So the total system becomes about 1.54 times faster

Now suppose the processor alone is improved by 2 times but processor execution accounts for only 30% of the total time:-

$$\text{Speedup}_{(\text{overall})} = \frac{1}{(1-0.3)+\frac{0.3}{2}} = \frac{1}{0.7+0.15} = \frac{1}{0.85} = 1.18$$

So the gain is only 1.18 times.

This shows that if the ERP slowdown is mainly due to memory delays, then improving only CPU speeds gives limited benifit. Therefore Amdahl's law suggests that cache and memory optimization provide better practical improvement than only upgrading the processor pipeline.

4. Whether processor, pipeline or cache improvement is better.

To improve the ERP performance, we must compare the impact of three major options

Improvement Area	Benifit	Limitation.
Processor upgrade.	faster execution	Expensive
Pipeline improvement.	Increases inst Throughput.	Hazards and branch penalties
Cache memory improvement	Reduces avg memory avg access time.	Limited by cache and locality

Processor improvement

A faster processor can execute arithmetic, logical and control instructions more quickly. However ERP systems are usually not dominated by heavy numerical computation alone. They involve

- → database queries
- → frequent memory access.
- → repeated instruction execution
- → branch-heavy workloads.
- → large-user transaction handling.

Because of this processor speed alone cannot fully solve the slowdown.

Pipeline Improvement

Instruction pipelining increases the number of instructions completed per unit time by overlapping instruction stages. This improves throughput significantly when many instructions are executed continously.

In ERP applications, pipelining is useful because many transactions involve repeated sequences such as

- → fetching records
- → checking conditions
- → performing arithmetic/logical operations.
- → storing updated data

Cache Improvement

ERP workloads repeatedly access

user data
database index tables
authentication records
frequently used code
temporary session information.

This means locality of reference is high. Therefore improving cache can reduce no. of slow main memory main memory accesses.

5. Instruction Pipelining and how it improves Execution Speed.

Instruction pipelining - a technique in which an instruction execution is divided into stages and multiple instructions are processed simultaneously at different stages.

· A common 5 stage pipeline is.

1. IF - Instruction fetch
2. ID - Instruction Decode
3. EX - Execute
4. MEM - Memory Access
5. WB - Write Back.

Without pipelining.

after the pipeline is filled. one instruction can complete in almost every cycle.

for 4 instructions in a 5 stage pipeline.

$$5 + (4-1) = 8 \text{ cycles}$$

to throughput improves greatly.

Inst 1	IF →	ID →	EX →	MEM →	WB
Inst 2	IF →	ID →	EX →	MEM →	WB
Inst 3	IF →	ID →	EX →	MEM →	WB
Inst 4	IF →	ID →	EX →	MEM →	WB

Practical relevance to ERP

The ERP server executes long instruction streams related to

- → request validation
- → query execution
- → condition checking.
- → result formatting.
- → memory update

So pipelining improves overall instructions throughput. and decreases average execution time.

6. Data Hazards and Control Hazards that Reduce Performance.

Although pipelining improves speed, it also introduces hazards

Data Hazards - happens when one instruction depends on the result of a previous instruction that has not yet completed.

Ex I1 ; ADD R1, R2, R3
I2 ; SUB R4, R1, R5

Instruction 2 needs the value of R1 produced by Inst 1. If pipeline executes them too closely. Instruction 2 may read the value old value of R1.

Solutions . operand forwarding / data forwarding.
pipeline stalling.
Instruction scheduling by compiler

Control Hazards - A control hazard occurs when the next instruction depends on a branch or jump decision.

Ex BEQ R1, R2, LABEL

until the processor knows whether the branch is taken or not, it may fetch the wrong next instruction.

ERP workloads often contain many.

- condition checks.
- loops
- validations
- decision statements.

Solutions

Branch prediction
Delayed branching.
Pipeline flushing when needed.
Smarter compiler scheduling

7. Practical solution for the college ERP server.

1. Improved cache memory

- increase L1/L2 cache efficiency.
- use better cache replacement strategy.
- optimize frequently accessed ERP code/data

2. Use pipelined processor support

if the existing processor is pipelined, optimize software to make better use of it.

reduce unnecessary dependencies and branches in software.

3. optimize memory access.

- improve locality of reference in ERP software.

4. Reduce hazard Impact.

- use branch prediction mechanisms
- Support forwarding to reduce stalls.

why this is Best

- full processor replacement is expensive
- memory stalls are often the major bottleneck in ERP workloads.
- Cache optimization gives immediate response time improvement.
- Pipelining improves throughput for repeated transactions.
- combined optimization gives the best balance of cost and performance.

8. Simple Performance View Using AMAT

a useful memory-performance formula.

AMAT = Hit Time + Miss Rate × Miss Penalty.

If cache improves AMAT decreases which directly improves ERP response time.

Example :- Suppose Hit time - 1 cycle
Miss rate - 5% = 0.05
miss penalty - 50 cycles

Then

$$AMAT = 1 + 0.05 \times 50$$
$$= 1 + 2.5 = 3.5 \text{ cycles}$$

if miss rate is reduced to 2%.

$$AMAT = 1 + 0.02 \times 50 = 1 + 1 = 2 \text{ cycles}$$

This is major improvement.

Final Recommendation

Prioritize cache and memory optimization.
support efficient pipelined execution
reduce data and control hazards.
avoid full hardware replacement unless unnecessary.

Since ERP systems heavily depend on memory access and repeated transaction processing, caching improvement combined with pipeline-aware execution is the most effective solution.

The slowdown of a college ERP server during peak usage is mainly caused by heavy instruction flow repeated accesses, and branch-heavy transaction handling.

Thus the recommended architecture solu is a cache optimized, pipeline processing environment with reduced stall and better memory access behaviour. This gives higher throughput, lower response time. etc

lower cost than full replacement and better suitability for real world ERP workloads.

Case Study -4 :- Cache Design for an Online Examination System

An online examination system is accessed simultaneously by thousands of students. During the exam, the system must quickly load questions, save answers, authenticate users, and process timing information without delay. If instruction and data fetches are slow, the response time of entire system degrades and may affect exam reliability.

The performance of such a system strongly depends on the memory hierarchy, especially cache memory, because the processor repeatedly accesses common instruction and data structures. Therefore, a well-designed cache system is necessary to reduce memory access time and ensure fast response.

Concepts from COA used in the Solution

The main concepts used here are:

- locality of reference.
- cache memory
- mapping techniques
- cache size, block size, and associativity trade-offs.
- write - through and write - back policies.
- Average Memory Access Time (AMAT)

These concepts help in designing a suitable memory hierarchy for a high-demand online examination system.

Role of Locality of Reference in Memory Design

Locality of reference means that programs tend to access the same memory locations repeatedly or access nearby memory locations within a short period of time.

Temporal Locality

If a memory location is accessed once, it is likely to be accessed again soon

Example in exam system:

- student session records
- time variable
- authentication data
- frequently executed login and save-answer routines

Spatial Locality

If one memory location is accessed, nearby locations are also likely to be accessed soon.

Example

- sequential instruction of the exam software.
- array elements of student responses.
- adjacent database entries.

Importance

Because of locality of reference, it is beneficial to keep recently used instruction and data in a small fast memory, called cache. This reduce the need to go to slower main memory repeatedly.

Purpose of cache memory and why it is faster than Main Memory

Cache memory is a small, high-speed memory placed between the CPU and main memory. its purpose is to store the most frequently used data and instructions.

Why Cache is faster

Cache is faster than main memory because:-

- It is built using SRAM.
- It is physically closer to the processor.
- access circuitry is simpler and faster.
- its size is much smaller than RAM.

Main memory usually uses DRAM, which is slower.

Benefit in Online Examination System

The exam platform repeatedly executes:

- login authentication code.
- question retrieval routines.
- answer submission routines.
- time update logic.
- session validation checks.

These repeated accesses can be served from cache, greatly improving response time.

Comparison of Mapping Techniques.

Cache mapping determines where a block of main memory can be placed in cache.

Direct Mapping

In direct mapping, each memory block can go to exactly one cache line.

Advantages

- simple
- fast
- low cost
- easy to implement

Disadvantages

- high conflict misses
- If two heavily used blocks map to the same line, they Keep replacing each other.

Suitability

- high conflict misses
- If two heavily used blocks map to the same line,
- Good for simple and low-cost systems, but not ideal for high-load online exam server because many active user may generate conflicting accesses.

Fully Associative Mapping

In a fully associative mapping any memory block can be placed in any cache line

Advantages
- lowest conflict misses.
- highest flexibility.

Disadvantages

expensive hardwardware
complex search logic
slower and costly for large caches

Suitability

Very efficient but often too expensive for large practical systems

Set associative Mapping.

In set associative mapping, the cache is divided into sets and a memory block can be placed in any line within its set.

Ex - 4-way set associative cache means each set has 4 possible locations.

Advantages

- good balance between speed and flexibility.
- fewer conflict misses than direct mapping.
- less costly than fully associative mapping.

Disadvantages

- more complex than Direct Mapping.
- moderate hardware cost

Suitability

This is best practical choice for an online examination System because it balances performance and cost.

Trade off Among Cache Size, Block Size, and Associativity

Cache performance depends on three important parameters

Cache Size

A larger cache can store more data and instructions

Advantage · reduces miss rate

Disadvantage · higher cost
· may slightly increase access time

In this case - a moderately large cache is useful because the exam server repeatedly uses common code and student session data.

Block Size - A block is the unit of data transferred from main memory to cache.

Larger Block Size Advantage

takes advantage of spatial activity.
useful when nearby data will be also be used.

Larger Block size Disadvantage

· may waste bandwidth if unnecessary data is loaded.
· fewer total blocks fit in cache.
· may increase miss penalty.

Associativity

Higher associativity reduces conflict misses.

Advantage

- better cache ulilization.
- fewer unnecessary replacements

Disadvantage

- more complex hardware.
- increased hit time

In this case - a 2 way or 4 way set associative cache is a strong choice.

Write-through vs Write Back Cache policies.

When the processor writes data, the cache must decide how to update main memory..

Write-Through

Every write updates both cache and main memory immediately.

Advantages

- simple
- memory always has latest data

Disadvantages

- more complex
  - risk of data loss if power failure occurs before update.
  - requires dirty bit handling.

Suitability - Better for high-performance systems with many write operations.

Recommended Policy for Online Examination System

For an Online exam System
- performance also matters under heavy load
- question reads are frequent
- answer submissions are important
- consistency and reliability matter greatly.

a good practical choice is

write back cache for better performance in server-side processing.

Combined with reliable backup/persistent saving mechanisms.

If the answer must be saved immediately and safely write ~~to~~ through is safer if higher performance is more critical and proper backup exists, write back is better.

a balanced recommendation is.

use write back in general cache for levels for speed

ensure final answer submission is committed reliably to main memory/storage.

Average Memory Access Time

AMAT = Hit Time + Miss Rate × Miss Penalty

Hit time = time to access cache
Miss Rate = fraction of accesses not found in cache
Miss Penalty = extra time to fetch from main memory

Suggested Memory Hierarchy Strategy

For the online examination system the recommended memory hierarchy is

1. CPU registers
2. L1 cache.
3. L2/L3 cache
4. main Memory (DRAM)
5. SSD storage.

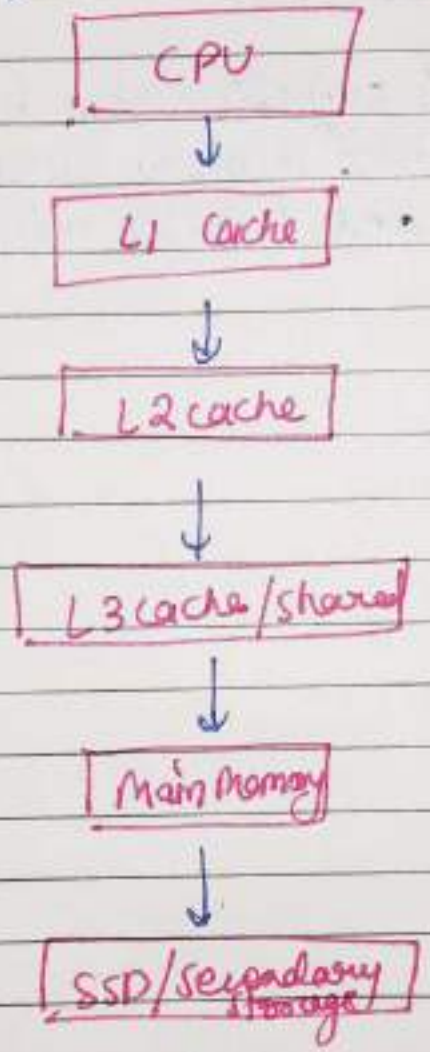
Recommended Cache Design

Set associative cache
Moderate-to-large cache size
Medium block size

Write-back policy with reliable commit support.

Strong use of locality-aware software design

Simple diagram of Memory Hierarchy.



Final Recommendation- The online examination platform should use a multilevel cache hierarchy with set associative mapping moderate cache size, medium block size, and an efficient write policy. This is because the system serves many repeated instructions and data requests making locality of reference important.

Among the direct mapping techniques.

direct mapping is simple. but may suffer from conflicts.

fully associative mapping is powerful but costly.

set associative mapping gives the best balance.

Therefore the most suitable design is a set associative multilevel cache-based memory hierarchy optimized for low AMAT and reliable write handling.